A1a: Slot Filling on ATIS with a CRF and a BiLSTM

Michael Incorvaia

September 2026

1 Setup

ATIS slot filling (4,478 / 500 / 893, 120 slots), strict span F1, every choice made on dev. The CRF (`sklearn-crfsuite`, c_1=c_2=0.1) uses word identity, affixes, shape, a ±1 template with bigrams, lower-cased words at ±2, and gazetteers plus a time regex (test 0.936). The BiLSTM is a 2-layer network with random 100-d embeddings and a handwritten CRF layer, trained at constant lr 10^{-3} (test 0.952); a softmax head is the ablation. Part C uses the fixed subsets of 224, 448, 1,120, and 4,478 sentences.

2 Part A: CRF feature engineering

Word identity alone (the surface form and its lower-cased form) is a dev F1 of 0.778 (0.776 test). On ATIS many strings are nearly unambiguous, so a memorized word-to-slot map already covers the easy spans. Prefixes and suffixes, then casing, digit, shape, and length, move dev F1 only to 0.783 and then 0.782. Those groups describe the form of the current token. On the 5% subset they help a little: identity alone is 0.662 and affixes raise it to 0.678. They do not choose a role. Role errors (right boundaries, right slot, wrong role, as in `toloc.city_name` versus `fromloc.city_name`) are 336 with identity alone and 337 once orthography is added. The affix row was not measured. Form features leave the from/to and depart/arrive decision where they found it.

The score arrives with context. The ±1 template and bigrams raise dev F1 from 0.782 to 0.928 and cut role errors from 337 to 101, because the preposition in "from Boston" versus "to Boston" is what chooses the role. The next row adds the lower-cased word at ±2. Case-folding of the current token is already in the identity group. Dev F1 moves from 0.928 to 0.951, which is enough to see a verb two tokens from a time ("leaving after noon"). Gazetteers and the time regex then reach 0.954, and role errors reach 60. Every list entry already occurs in the full training set, so those features barely move full-data F1 (bottom block of Table 1). On 224 sentences the missing list items are 13 of 15 airport codes, 11 of 23 airline words, and 10 of 19 day words. Three of the 34 cities are missing. Removing the lists costs 0.853 down to 0.835. Removing context returns the model to 0.783.

3 Part C: Data efficiency

On test the curves cross between 10% and 25% of the training data. The CRF leads at 224 sentences (0.847 vs. 0.816) and at 448 (0.887 vs. 0.883); the BiLSTM-CRF leads at 1,120 (0.919 vs. 0.905) and at 4,478 (0.952 vs. 0.936). Dev flips earlier, at 448 (0.901 vs. 0.891). Below the crossover, gazetteers, shape, and the ±1 templates already supply regularities the randomly initialized encoder has not learned: the 0.816 ±0.011 band stays under 0.847. Above it, the encoder covers dependencies outside that window, such as a verb several tokens from its time. On dev, switching from ReduceLROnPlateau to the constant learning rate moves the crossover from between 10% and 25% to between 5% and 10%. On test both schedules still cross between 10% and 25%. The constant rate shrinks the 10% test gap from 0.028 (0.887 vs. 0.859) to 0.004 (0.887 vs. 0.883). Full-data test F1 barely changes (0.951 to 0.952).

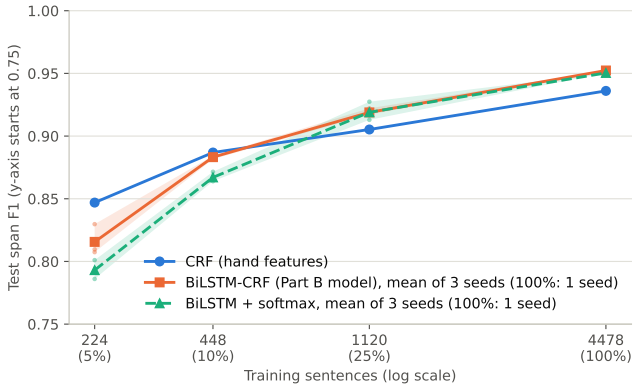


Figure 1: Test span F1 vs. training-set size (log scale). BiLSTM curves are the mean of 3 seeds at 5–25% (band = min–max, dots = individual seeds; ± in Table 3 is half that range) and 1 seed at 100%. CRF training is deterministic. The plotted BiLSTM uses a constant learning rate chosen on dev, replacing a ReduceLROnPlateau schedule that undertrained the small subsets (test F1 0.798 at 5%, 0.951 at 100%).

The CRF output layer is worth the most while the encoder is weak: 0.816 vs. 0.793 at 5%, where the transition matrix blocks illegal BIO paths. The two heads tie at 25% (0.919 and 0.919) and the full-data gap is 0.952 vs. 0.950. The win over the feature CRF is the encoder. The structured output is a small-data constraint.

4 Part D: Error analysis

Most remaining errors are the wrong type: 142 for the CRF and 102 for the BiLSTM-CRF, against 12 and 11 boundary errors (Table 4). The models find the span and then assign the wrong role, the same gap ±1 context closed in the feature log (336 role errors down to 101). The BiLSTM-CRF misses fewer spans (21 vs. 33). The CRF alone is wrong on 50 sentences, the BiLSTM-CRF alone on 24, and both on 90. Recall on spans with an unseen

Table 1: CRF feature log (last column: same features trained on the 5% subset). Top block adds feature groups cumulatively; bottom block removes one group from the full set.

	Trained on 100% (4,478 sent.)				5% (224)
Feature set	#feats	Dev F1	Test F1	Role errs	Dev F1
word identity only (current token)	2,966	0.778	0.776	336	0.662
+ prefixes/suffixes (1–3 chars)	7,338	0.783	0.771	–	0.678
+ casing, digit, shape, length	8,270	0.782	0.776	337	0.675
+ all of the above for tokens ±1, bigrams	20,082	0.928	0.921	101	0.832
+ lower-cased words at ±2	19,922	0.951	0.933	–	0.835
+ gazetteers / time regex (**submitted**)	18,596	**0.954**	0.936	60	**0.853**
full − gazetteers	19,922	0.951	0.933	–	0.835
full − affixes	11,919	0.953	0.935	–	0.834
full − orthographic	16,946	0.953	0.934	–	0.851
full − all context	8,065	0.783	0.776	–	0.696

#feats = state features with nonzero weight after training; the L1 penalty (c_1=0.1) prunes features made redundant by others, so the count can drop when a group is added. Role errs = dev spans with the right boundaries and the right slot but the wrong role (e.g. `toloc` vs `fromloc`), out of 1,709 gold spans; – = not measured.

Table 2: Ten test failures. Bold = disputed tokens; † = token never seen in training. "Wrong" says which model errs. Index in parentheses is the test line.

#	Wrong	Utterance, then the disputed spans (G = gold, C = CRF, L = BiLSTM-CRF)
D1 (176)	CRF	what meals are available on dl 468† which al† arrives in san francisco at **950† am** G: `arrive_time.time`=950 am C: `depart_time.time`=950 am L: `arrive_time.time`=950 am
D2 (368)	CRF	how many passengers can an **l1011†** aircraft hold† G: `aircraft_code`=l1011 C: – L: `aircraft_code`=l1011
D3 (359)	CRF	what is **phl†** G: `airport_code`=phl C: – L: `airport_code`=phl
D4 (512)	CRF	what is **tpa†** G: `airport_code`=tpa C: – L: `airport_code`=tpa
D5 (204)	LSTM	what is the first class round trip airfare from **india†** indianapolis to memphis G: – C: – L: `fromloc.city_name`=india
D6 (773)	LSTM	how many people will a **757 hold†** G: `aircraft_code`=757 C: `aircraft_code`=757 L: `transport_type`=757 hold
D7 (164)	LSTM	how many canadian airlines international flights use aircraft **320†** G: `aircraft_code`=320 C: `aircraft_code`=320 L: –
D8 (165)	LSTM	how many canadian airlines flights use aircraft **dh8†** G: `aircraft_code`=dh8 C: `aircraft_code`=dh8 L: –
D9 (51)	both	what meals are served on american flight **665† 673†** from milwaukee to seattle G: `flight_number`=665 673 C: `flight_number`=665; `flight_number`=673 L: `flight_number`=665
D10 (243)	both	what is the seating capacity for **delta be1†** G: `aircraft_code`=be1; `airline_name`=delta C: `airline_name`=delta be1 L: `airline_name`=delta; `flight_number`=be1

Table 3: Span F1 behind Fig. 1. ± is half the min–max range over 3 seeds. Best test score per row in bold.

		Test span F1			Dev span F1
Frac.	Sent.	CRF	BiLSTM-CRF	BiLSTM+softmax	CRF / BiLSTM-CRF
5%	224	**0.847**	0.816 ±0.011	0.793 ±0.008	0.853 / 0.848
10%	448	**0.887**	0.883 ±0.001	0.867 ±0.004	0.891 / 0.901
25%	1,120	0.905	0.919 ±0.002	**0.919** ±0.007	0.925 / 0.948
100%	4,478	0.936	**0.952**	0.950	0.954 / 0.979

Table 4: Test-set error profile of the two 100% models.

Span errors on test (2,837 gold spans)	CRF	BiLSTM-CRF
wrong type	142	102
wrong boundary	12	11
type + boundary	8	7
missed	33	21
spurious	6	10
recall, spans w/ OOV token (n=50)	0.540	0.620
recall, in-vocab spans (n=2787)	0.938	0.956
sentences wrong (this model only)	50	24
sentences wrong for both	90	

token is 0.540 vs. 0.620 (50 spans), against 0.938 and 0.956 in vocabulary. The CRF leads on `aircraft_code` (0.935 vs. 0.653, n=33) from word identity, digit/shape, and the word "aircraft" beside the code. There is no aircraft-code gazetteer. It also leads on `fromloc.airport_code` (0.909 vs. 0.769, n=5), where the airport-code list can fire. The BiLSTM-CRF leads on bare `airport_code` (0.444 vs. 0.200, n=9) and on compositional time and date roles, including `arrive_time.period_of_day` (0.857 vs. 0.000, n=6). The period list contains "morning" and "evening," and the CRF's F1 on that slot is still zero: the list marks

Table 5: Slot types ($n \geq 5$) with the largest F1 gap.

Slot type	n	CRF	BiLSTM-CRF	Δ
`aircraft_code`	33	0.935	0.653	-0.282
`fromloc.airport_code`	5	0.909	0.769	-0.140
`arrive_time.start_time`	8	0.750	0.941	+0.191
`arrive_time.time`	34	0.746	0.943	+0.197
`airport_code`	9	0.200	0.444	+0.244
`arrive_date.month_name`	6	0.286	0.769	+0.484
`arrive_date.day_number`	6	0.286	0.769	+0.484
`arrive_time.end_time`	8	0.200	0.800	+0.600
`arrive_date.day_name`	11	0.235	0.880	+0.645
`arrive_time.period_of_day`	6	0.000	0.857	+0.857

a period, and the arrive/depart role is left to context.

Errors only the CRF makes (D1–D4).

D1. Gold and the BiLSTM-CRF mark "950 am" as `arrive_time.time`; the CRF emits `depart_time.time`. The time regex fires on the unseen digits and "am" is in the period list, but "arrives" (token 9) is five positions from "950" (token 14), outside the ± 1 template and the ± 2 words. The BiLSTM's state at the time still contains the verb.

D2. Gold and the BiLSTM-CRF mark unseen "l1011" as `aircraft_code`; the CRF emits nothing. Shape `xdddd` sets neither `isdigit` nor the time regex, and there is no aircraft-code list. The BiLSTM sees `<UNK>` between "an" and "aircraft" and labels it from that frame.

D3 and D4. "phl" and "tpa" are unseen and absent from the airport-code list, which keeps only codes from training. Gold is `airport_code`. The BiLSTM-CRF gets both, since both map to `<UNK>` in "what is `<UNK>`." The CRF has no identity feature and no list hit, so it misses both.

Errors only the BiLSTM makes (D5–D8).

D5. Gold leaves unseen "india" as O, a restart before "indianapolis." The city list does not contain "india," so the CRF also emits O. The BiLSTM sees `<UNK>` after "from" and emits `fromloc.city_name`.

D6. "757" is in the training vocabulary; "hold" is unseen. Gold and the CRF mark a one-token `aircraft_code`, from word identity and `isdigit`. The BiLSTM pulls `<UNK>` into the span and emits `transport_type` on "757 hold."

D7 and D8. Unseen "320" and "dh8" sit immediately after "aircraft." Gold and the CRF mark `aircraft_code` from that previous-word feature; the BiLSTM sees `<UNK>` and emits O. This is the reverse of D2, and it is why `aircraft_code` is the CRF's largest gap in Table 5.

Errors both make (D9–D10).

D9. Gold is one `flight_number` span on unseen "665 673." The CRF emits two spans, one per digit string, and the BiLSTM keeps only "665." `isdigit` and the time regex fire on each token separately, so neither model joins the unseen pair.

D10. Gold is `airline_name` on "delta" and `aircraft_code` on unseen "be1." The CRF merges "delta be1" into one airline span. The BiLSTM splits them and labels "be1" as `flight_number`, the usual short code after an airline name.

5 Would I have switched to neural in 2014?

With only these results, I would ship the BiLSTM-CRF on the full training set and keep the linear CRF below the test crossover, between 448 and 1,120 sentences. At 4,478 the neural model leads 0.952 to 0.936, 1.6 points. That is the scale of the last two feature steps on dev, from 0.928 to 0.954 (2.6 points). The ± 1 context step, 0.782 to 0.928, was 14.6 points. The gain is uneven: long-range roles and compositional dates and times, plus OOV recall 0.620 vs. 0.540, while `aircraft_code` stays with the CRF (0.935 vs. 0.653) via identity, shape, and the word "aircraft." The constant learning rate had to be chosen on dev, where it does move the crossover. On test it does not, and the old schedule was already ahead at full data (0.951). The 5% seed band is about a point. I would have shipped the BiLSTM-CRF at 4,478 after locking that learning rate, and left the feature CRF in place for equipment codes and for the small side of the test crossover. Mesnil et al. (2015) made that comparison for ATIS: recurrent taggers passing a CRF [1].

References

[1] Mesnil, G., Dauphin, Y., Yao, K., Bengio, Y., Deng, L., Hakkani-Tur, D., He, X., Heck, L., Tur, G., Yu, D., and Zweig, G. 2015. Using recurrent neural networks for slot filling in spoken language understanding. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 23(3):530–539.

AI-use disclosure

-
- Code review: an AI assistant (Claude) compared the repo with the course skeleton, simulated the autograder's inference check on the submission zip, and checked the Viterbi decoder and CRF loss against brute-force enumeration.
- Experiments run by the assistant at my request: the CRF feature ablation (`crf_ablation.py`), removing two test-only airport codes from the gazetteer and retraining the CRF, extra BiLSTM seeds, the softmax-only BiLSTM (`-no-crf`), and the constant-lr BiLSTM recipe that replaced the submitted checkpoint after it scored higher on dev.